# 山东大学 计算机科学与技术 学院

## 机器学习 课程实验报告

| 学号：202200130077 | 姓名： 薛润泽 | 邮箱： |
|---|---|---|
| 实验题目：Experiment 9: Decision Tree | | |

实验过程：

1、决策树的构建

（1）选择使用 Information Gain 来评价最优属性 best_feature 的选择，以及数据集分裂点的选择。Information Gain 原理如下图：

$$Gain(S,A) = E(S) - I(S,A) = E(S) - \sum_i \frac{|S_i|}{|S|} E(S_i)$$

其中，E（S）是当前数据集的 Entropy 熵值，I（S,A）可以看做是所有分类下的熵值的均值。所以 Information Gain 是通过熵值 Entropy 来计算的。

Entropy 计算公式如下：

$$E(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-$$

这样，我们就有了建立决策树的一个标准，同样这个标准适用于最优属性选择和数据集分裂。

（2）选择使用元组的形式来存储决策树节点信息，节点信息以如下存储：

[sym , feature , feature data , left_node, right_node ]

其中，sym 记录这个节点是一个过程节点还是叶子节点。如果 sym 值是 2，则是一个过程节点，在决策树进行分类时要继续递归查找；如果 sym 值是 0 或者 1，说明这个节点是一个叶子节点，如果值为 0 说明分类结果为负类，如果值为 1 说明分类结果为正类；

Feature 用来记录该点应该通过什么属性进行分类，feature data 决定了测试点是继续向左子树查找分类还是右子树查找分类。在我的实现中，默认测试点属性如果小于等于 feature data，则向左子树查找；如果大于 feature data，则向右子树查找。

Left_node, right_node 等记录该节点的左右子树根节点在整棵决策树中的索引位置。

决策树自身使用列表来进行存储。

（3）为了防止过拟合，在建树的过程中使用了前剪枝策略

简单起见，我使用 Entropy 熵来作为是否进行前剪枝的判断依据。在分裂阶段，计算一下本数据集的熵，如果这个熵小于某个值，则我们认为精度可以满足我们的需求，且误差我们可以接受，不需要继续分类了，直接将该数据集所在点作为叶子节点。在本次实验中，熵阈值我选择了 0.2

（4）决策树的递归建立，在建立过程中，注意停止递归的条件，包括所在数据集数据标签全部相同，所在数据集最优分类属性取值全部相同，所在数据集为空，所在数据

集熵值小于熵阈值等等。因为 Python 中的传参为引用传参，所以可以方便的将新节点加入到记录整棵树的列表中去。

2、测试集的分类结果预测

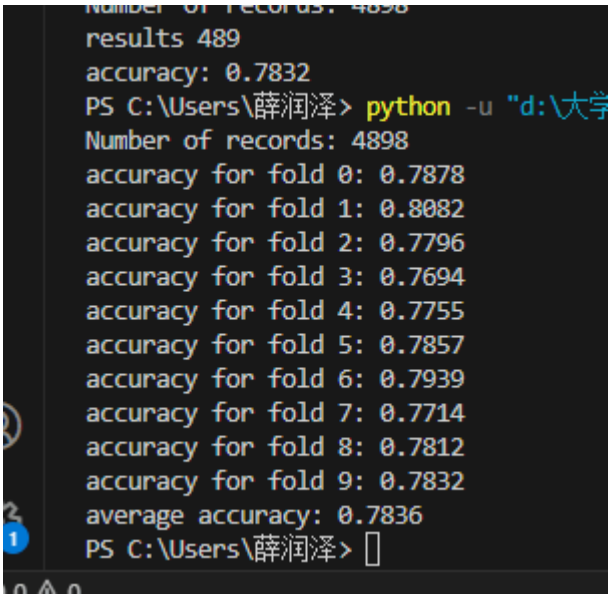
（1）递归式进行测试数据点的分类结果预测，因为我们树上的每个点的形式为：

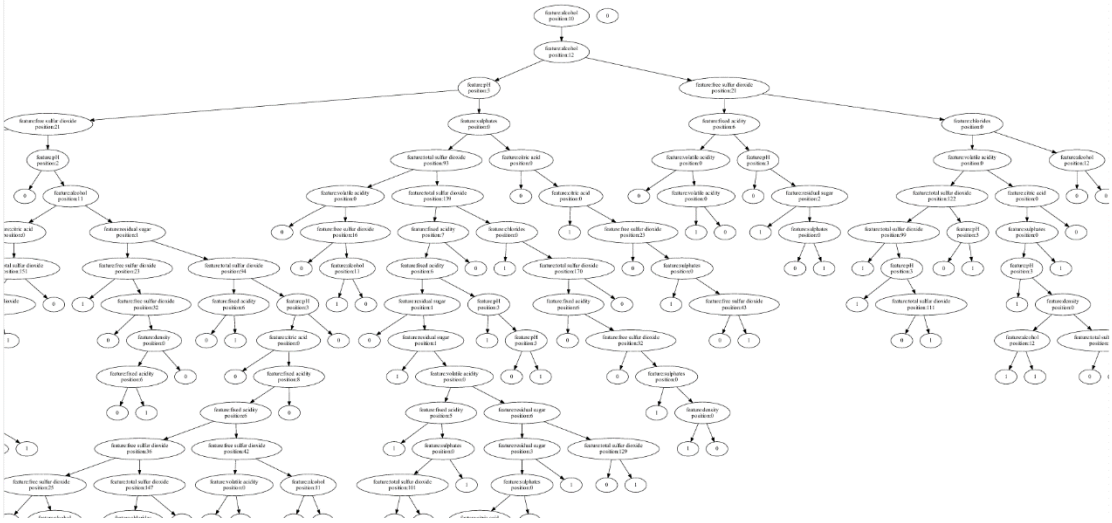
[sym , feature , feature data , left_node, right_node ]

所以，直到遍历到的点的 sym == 1 或者 sym == 0 时结束递归

（2）交叉验证法这里选择将总数据集分成 10 分来交叉验证。交叉验证的 accuracy 结果上，我选择使用 10 个数据集的平均值作为最后的准确率。

3、结果分析：



可以看到，平均准确度达到 78.36%，高于 78.32%，决策树起到了作用

从决策树图上看，只做前剪枝树依旧比较复杂

5、实际代码（注意：运行时要先转到本机工作区）

```
# CSE6242/CX4242 Homework 4 Sketch Code
# Please use this outline to implement your decision tree. You can add any
code around this.

import csv
import os
import math

from graphviz import Digraph

os.chdir("D:/大学/大二下/机器学习/实验/ex9/EX9PY")#转到本机工作区
# Enter You Name Here
myname = "Xue-Runze-" # or "Doe-Jane-"

# Implement your decision tree below
def Entropy(data):# data 是一个 0, 1 向量
    one_num = 0
    one_num =int(sum(data))
    zero_num = len(data) - one_num
    one_calc = 0
    zero_calc = 0
    if one_num != 0:
        one_calc = 1.0 * one_num / len(data) * math.log2(1.0 * (one_num) /
len(data))
    if zero_num != 0:
        zero_calc = 1.0 * zero_num / len(data) * math.log2(1.0 * (zero_num) /
len(data))
    return  -one_calc - zero_calc

def Entropy_2(zero_num, one_num):
    total_num = zero_num + one_num
    one_calc = 0
    zero_calc = 0
    if one_num != 0:
        one_calc = 1.0 * one_num / total_num * math.log2(1.0 * (one_num) /
total_num)
    if zero_num != 0:
        zero_calc = 1.0 * zero_num / total_num * math.log2(1.0 * (zero_num) /
total_num)
    return  -one_calc - zero_calc

def is_same(data):#判断数据标签是否为相同标签
    sym = 1
    check = data[0][11]
```

```
    for i in data:
        if i[11] != check:
            sym = 0
    return sym

def select_best_feature(data): #找到最佳分界点  // 前 11 行属性 第 12 行标签
    best_feature = 0  #属性号
    best_data = 0
    t_z_num = 0
    t_o_num = 0
    label_data = [] #0 1 序列点
    for it in data:
        label_data.append(it[11])
        if abs(it[11] - 1) <= 0.0001:
            t_o_num = t_o_num + 1
        elif abs(it[11]) <= 0.0001:
            t_z_num = t_z_num + 1
    Plus_sum = t_o_num + t_z_num # 1 , 0 总个数
    Eorigin = Entropy(label_data)
    record_divide = [0,0,0,0,0,0,0,0,0,0,0]
    for i in range(0, 10):
        record_divide[i] = 0# record_divide 初始化
    for index in range(0, 11): #前 11 个属性
        sorted_data = sorted(data, key = lambda x:x[index])
        zero_num = 0
        one_num = 0
        divide_sym = sorted_data[0][index] #存储分离点实际值，只存储每个属性计算
出最大的 IG 的分离点
        for label_index in range(0, len(sorted_data)):
            if sorted_data[label_index][11] == 1:
                one_num = one_num + 1
            elif sorted_data[label_index][11] == 0:
                zero_num = zero_num + 1
            if sorted_data[label_index][index] == divide_sym:
                continue
            divide_sym = sorted_data[label_index][index]
            T_IG = 1.0 * (one_num + zero_num) / Plus_sum *
Entropy_2(zero_num, one_num) + 1.0 * (Plus_sum - one_num - zero_num ) /
Plus_sum * Entropy_2(t_z_num - zero_num, t_o_num - one_num)
            if Eorigin - T_IG > best_data:
                best_data = Eorigin - T_IG
                best_feature = index
                record_divide[index] = label_index #记录分割点索引下标
```

```
    sorted_data = sorted(data, key = lambda x:x[best_feature]) #按照最优属性再排一下序，取出存入 record_divide[]中的分割点 索引

    divide_math = int(record_divide[best_feature]) + 1
    left_sub_list = sorted_data[:divide_math]
    right_sub_list = sorted_data[divide_math:]
    zero_or_one = 2
    if zero_num > one_num :
        zero_or_one = 0
    else:
        zero_or_one = 1
    return [left_sub_list, right_sub_list, best_feature ,sorted_data[int(record_divide[best_feature])][best_feature], Eorigin, zero_or_one] #第三个元素是 对应属性的具体值

def BuildTree(data, Tree_list, cnt_index):

    if len(data) == 0 :
        Tree_list[cnt_index] = (3,-1)
        return None
    if is_same(data):
        if data[0][11] == 1:
            Tree_list[cnt_index] = (1,-1)
        else :
            Tree_list[cnt_index] = (0, -1)
        return

    [left_sub_list, right_sub_list, best_feature, spilit_value, Eorigin, one_or_zero] = select_best_feature(data)
    if Eorigin <0.2:
        Tree_list[cnt_index] = (one_or_zero, -1)
        return

    #这里有多个前剪枝策略，所需值 存入第 4 个返回元素中， 我先选择使用 熵大小来判断是否前剪枝，如果熵小于 0.2 则直接划归叶子结点
    if Eorigin <= 0.2:
        Tree_list[cnt_index] = (one_or_zero, -1)
        return
    left_index = len(Tree_list)
    right_index = len(Tree_list) + 1
    Tree_list.append(left_index)
    Tree_list.append(right_index)
    Tree_list[cnt_index] =(2, best_feature, spilit_value, left_index, right_index)
```

```
    if(len(left_sub_list) > 0):
        BuildTree(left_sub_list, Tree_list, left_index)
    if(len(right_sub_list) > 0):
        BuildTree(right_sub_list, Tree_list, right_index)

class DecisionTree():
    Tree = []
    def learn(self, training_set):
        # implement this function
        BuildTree(training_set, self.Tree, 0)
    # implement this function
    def classify(self, test_instance):
        result = 0 # baseline: always classifies as 0
        root = self.Tree[0] #root 为 sym 0, feature 1, feature data 2,
left_node 3, right_node 4
        while root[0] == 2:
            if test_instance[root[1]] <= root[2]:
                root = self.Tree[root[3]] #向左子树找
            else:
                root = self.Tree[root[4]] #向右子树找
        if root[0] == test_instance[10]:
            result = 1
        return result

def run_decision_tree():
    # Load data set
    with open("ex6Data.csv") as f:
        next(f, None)
        data = [tuple(float(item) for item in line) for line in csv.reader(f,
delimiter=",")]
    print ("Number of records: %d" % len(data))

    # 10-fold cross-validation
    K = 10
    accuracies = []  # list to store accuracy for each fold

    for k in range(K):
        training_set = [x for i, x in enumerate(data) if i % K != k]
        test_set = [x for i, x in enumerate(data) if i % K == k]

        tree = DecisionTree()
        # Construct a tree using training set
        tree.learn(training_set)
```

```
        # Classify the test set using the tree we just constructed
        results = []
        for instance in test_set:
            result = tree.classify(instance[:-1])
            results.append(result == instance[-1])

        # Accuracy
        accuracy = float(results.count(True))/float(len(results))
        print ("accuracy for fold %d: %.4f" % (k, accuracy))
        accuracies.append(accuracy)

    # Average accuracy
    # print(sum(accuracies))
    avg_accuracy = sum(accuracies) / len(accuracies)
    print("average accuracy", avg_accuracy)
    # Writing results to a file (DO NOT CHANGE)
    f = open(myname+"result.txt", "w")
    f.write("average accuracy: %.4f" % avg_accuracy)
    f.close()

if __name__ == "__main__":
    run_decision_tree()

# 假设你的决策树对象是 tree
```

图形生成的示例代码：

```
from graphviz import Digraph

def draw_tree(tree, dot=None):
    if dot is None:
        dot = Digraph()
        dot.node(name=str(tree), label=str(tree.value))

    for child in tree.children:
        if child is not None:
            dot.node(name=str(child) ,label=str(child.value))
            dot.edge(str(tree), str(child))
            draw_tree(child, dot)

    return dot

# 假设你的决策树对象是 tree
dot = draw_tree(tree)
dot.render('decision_tree.gv', view=True)  # 保存并打开生成的图
```

**结果分析与体会：**

结果与预期接近。

决策树在建立过程中要做好前剪枝和后剪枝，防止决策树过拟合。

决策树递归建树的过程中，如果代码出现错误，容易造成深度过深，python 主动停止运行。错误类型可能有，数据集分割失败，最优属性选择出现问题，递归终止条件有误等。